

KAOS — Architecture & Component Report

KAOS = Kafka Automated Ops System. An event-driven, closed-loop CI/CD orchestration platform where autonomous AI agents manage the software lifecycle — specifically the *unhappy paths* (bugs, rejections, merge conflicts, deploy failures) — with minimal human intervention.

This document explains **what each component is, how it is built, and why it was built that way**. It is written so you can defend every design choice in an interview.

1. The One-Paragraph Pitch

KAOS watches your engineering org's external signals — Sentry errors, GitHub PR webhooks, and deployment status — and reacts to them automatically. A bug comes in → an agent finds the right owner from an org graph, files a Jira ticket, logs it in Notion, and DMs the owner on Slack. A PR is opened → an agent assigns a qualified reviewer. A review is rejected → the author is pinged with the comments. A deploy fails → an LLM diagnoses the logs, notifies the author *and* reviewer, and **re-injects the failure as a new bug report**, closing the loop back to triage. Every step is recorded in an event store that a chatbot can later narrate ("What happened with PaymentService?").

2. Architecture at a Glance

2.1 Component map



2.2 The "3-Agent Model" (plus 2 support services)

Concerns are **strictly separated by lifecycle phase** to avoid state leakage:

Agent	Phase	Trigger topic	Core responsibility
Triager (Agent 1)	Intake	<code>system.quality.reports</code>	Route a failure to the right human
Review Manager (Agent 2)	Development loop	<code>dev.pr.updates</code> , <code>dev.pr.decisions</code>	Shepherd the PR; handle rejections/conflicts. <i>Never touches prod.</i>
Ops Manager (Agent 3)	Deployment	<code>ops.deploy.status</code>	Manage go-live + post-deploy health; diagnose failures
Ingestion (support)	Edge	— (HTTP in)	Translate noisy external webhooks → clean internal Kafka events
Chatbot (Agent 4)	Observability	— (HTTP in)	Answer "what happened?" from the event store
Control Plane (support)	Ops/Demo	— (HTTP in)	Process orchestration + simulation + UI

Why three agents and not one monolith? Each phase has a different trust boundary and blast radius. The Review Manager is explicitly forbidden from touching production; the Ops Manager is the only thing allowed to. Splitting them means a bug in PR-routing logic can never accidentally trigger a deploy, and each agent can be scaled, restarted, and reasoned about independently. It also maps cleanly onto Kafka consumer groups — each agent is its own group with its own offset cursor.

3. The Backbone: Event-Driven Design

3.1 Why Kafka (and why these four topics)

The whole system is **choreographed, not orchestrated**. There is no central "manager" calling each agent in sequence. Instead, agents publish facts to topics, and other agents react. This is deliberate:

- **Decoupling** — Ingestion does not know Triager exists. It just publishes a `QualityReport`. You can add a second consumer (e.g., a metrics dashboard) on the same topic without touching producers.
- **Durability & replay** — Kafka retains events. `auto.offset.reset='earliest'` (see `shared/kafka/client.py`) means a freshly-started agent can replay history. Restarting an agent resumes from its committed offset rather than losing work.
- **Back-pressure & buffering** — A burst of 500 Sentry errors queues in Kafka; agents drain at their own pace instead of falling over.
- **Natural audit log** — The topic *is* the history of what the org experienced.

The **four topics form a taxonomy by semantic intent**, not by source system (`reference/TOPICS.md`):

Topic	Meaning	Producer	Consumer
<code>system.quality.reports</code>	"Something is broken"	Ingestion, Ops Manager (loop)	Triager
<code>dev.pr.updates</code>	"Code changed" (open/sync/merge)	Ingestion	Review Manager
<code>dev.pr.decisions</code>	"A human/AI decided" (approve/reject)	Ingestion	Review Manager
<code>ops.deploy.status</code>	"Deployment lifecycle event"	Ingestion	Ops Manager

Splitting `dev.pr.updates` from `dev.pr.decisions` matters: *updates* are high-frequency mechanical events (every push), while *decisions* are low-frequency, high-importance human signals. Different topics let you tune retention, partitioning, and consumer logic independently.

3.2 The Kafka client (`shared/kafka/client.py`)

A thin factory over `confluent_kafka`. It centralizes the SASL_SSL connection config (Confluent Cloud uses `SASL_SSL` + `PLAIN`) so no agent re-implements auth. Consumers get `group.id` + `auto.offset.reset='earliest'`. **Why a factory and not a singleton client?** Producers and consumers have different lifecycles; the factory lets each agent ask for exactly what it needs while sharing one source of credential truth.

3.3 The consumer base class (`shared/kafka/consumer_base.py`)

An abstract base (`BaseAgentConsumer`) implementing the **Template Method pattern**: it owns the boilerplate (subscribe, poll loop, JSON decode, error handling, partition assign/revoke callbacks, graceful shutdown) and leaves a single `process_message(self, message)` abstract hook for each agent to fill in.

- **Why:** Every agent's *infrastructure* loop is identical; only the *business logic* differs. This removes ~40 lines of duplicated polling code per agent and guarantees consistent error handling (a `JSONDecodeError` in one agent is handled exactly like in every other).
- **At-least-once semantics:** it relies on Kafka's default auto-commit. The code comments are honest that production might want manual commits after successful processing for stronger guarantees — a great thing to discuss in an interview (see Q on delivery semantics).

4. The Brain: Neo4j Org Graph

4.1 Why a graph database

The central question KAOS keeps asking is **relationship-shaped**: "Who owns this service? If they're unavailable, who worked on it? If nobody, who do they report to?" Those are graph traversals. In SQL you'd write recursive joins; in Cypher they're one-liners.

4.2 Schema (`reference/SCHEMA.md` , `docs/graph_schema.md` , seeded by `scripts/seed_neo4j.py`)

Nodes

- `(:Person)` — `name` , `role` (Junior/Senior/Lead/External), `status` (**Active / On_Leave / Inactive** — the key routing property), `slack_id` , `email`
- `(:Service)` — `name` , `repo_url` , `tier` (1 = critical)
- `(:Team)` — `name` , `slack_channel`

Relationships (the actual logic lives here)

- `(:Person)-[:OWNS]->(:Service)` — primary owner; Triager checks this first
- `(:Person)-[:WORKED_ON]->(:Service)` — contributor; fallback when owner is away
- `(:Person)-[:REPORTS_TO]->(:Person)` — escalation path
- `(:Service)-[:BELONGS_TO]->(:Team)` — routes alerts to the right Slack channel

4.3 The "active routing" algorithm (`agents/triager/tools.py :: find_service_owner`)

This is the system's signature piece of intelligence, a **three-tier fallback**:

1. **Direct active owner** — `OWNS` + `status = 'active'` .
2. **Active contributor** — anyone who `WORKED_ON` the service and is active. (Owner is on leave? Hand it to someone who's touched the code.)
3. **Escalation to manager** — follow `REPORTS_TO` .

Two design details worth defending:

- **Name normalization:** `payment-service` , `PaymentService` , and `payment_service` all collapse to `paymentservice` via `toLowerCase(replace(replace(name, '-', ''), '_ ', ''))` . External systems are inconsistent about casing; the graph match must be tolerant.
- **Status-first matching:** the query filters on `status = 'active'` *inside* Cypher rather than fetching the owner and checking in Python. This means an on-leave owner is never even returned — routing is correct by construction.

The Review Manager's `find_reviewer` is the mirror image: "find a **Senior, Active** engineer who **OWNS** the service but is **NOT** the PR author" — load-balancing reviews to qualified people while preventing self-review.

4.4 The Neo4j client (`shared/neo4j/client.py`)

Deliberately minimal: a driver wrapper with one `query(cypher, params)` method that runs inside a session and returns `.data()` . **Why so thin?** The intelligence is in the Cypher, not the client. Keeping the client dumb means queries live next to the business logic that owns them (in each agent's `tools.py`), which is easier to reason about than a fat repository layer.

5. The Agents in Detail

A crucial architectural nuance: **KAOS uses two execution styles**, and choosing between them per-agent is itself a design decision.

Style	Used by	How it works	Why
Deterministic pipeline	Triager <code>consumer.py</code> , Review Manager <code>consumer.py</code>	Hard-coded Python calls the tools in a fixed order	These workflows are well-defined state machines. You don't want an LLM "creatively" skipping the Jira step. Reliability > flexibility.
LLM ReAct agent	Ops Manager <code>main.py</code> , Chatbot <code>main.py</code>	<code>BaseAgent</code> reasons and picks tools	Failure diagnosis and free-text Q&A are open-ended. Here you <i>want</i> the model to decide which tools to call.

The repo contains *both* a `main.py` (LLM version) and a `consumer.py` (deterministic version) for Triager and Review Manager. The Control Plane runs the `consumer.py` variants in production — the LLM `main.py` files are the original/alternative design kept for reference. Being able to explain this trade-off ("we prototyped with an LLM loop, then hardened the predictable paths into deterministic code") is a strong interview talking point.

5.1 Ingestion Agent (`agents/ingestion/main.py`) — the Anti-Corruption Layer

A FastAPI service (port 8000) exposing three webhook endpoints:

- `/webhooks/sentry` → maps Sentry's `level` (error/fatal/warning) to internal `severity` (HIGH/CRITICAL/MEDIUM) → publishes `QualityReport` to `system.quality.reports` .
- `/webhooks/github` → inspects the `X-GitHub-Event` header + `action` to distinguish PR open/sync/merge (→ `dev.pr.updates`) from reviews (→ `dev.pr.decisions`).
- `/webhooks/deployment` → maps `success/failure` → `SUCCEEDED/FAILED` → `ops.deploy.status` .

Why a dedicated ingestion service?

- Anti-corruption layer (DDD):** External payloads are messy and change without warning. Ingestion is the *one* place that knows Sentry's/GitHub's wire format. Everything downstream consumes clean, Pydantic-validated internal events. If GitHub changes its schema, you patch one file.
- Non-blocking publish:** It uses FastAPI `BackgroundTasks` to push to Kafka *after* returning `200` to the webhook caller. Webhook senders (GitHub) time out fast and retry aggressively; you must ack immediately.
- Validation at the edge:** Pydantic models in `agents/ingestion/models.py` (`QualityReport` , `PRUpdate` , `PRDecision`) reject malformed input before it ever reaches Kafka.

5.2 Triager (Agent 1) — `agents/triager/consumer.py`

On each `system.quality.reports` event:

- Resolve assignee.** If the event carries a `suggested_assignee` (the closed-loop case from Ops Manager), use it directly. Otherwise call `find_service_owner` (the 3-tier graph routing).

2. **Recurring-issue check.** Query the local SQLite store (`get_active_jira_ticket(service)`). If an active ticket already exists, **update** it ("⚠️ Recurring Incident") instead of creating a duplicate, and return early. This is the **idempotency guard**.
3. **Create Jira ticket** (`create_jira_ticket`) — maps severity→priority, assigns the user, leaves it in "To Do".
4. **Log to Notion** (`add_to_notion_dashboard`) — creates a dashboard row with status "Open".
5. **Notify on Slack** (`send_slack_message`) — DMs the assignee *and* announces in `#all-kaos` .
6. **Record** a `TICKET_CREATED` event to the event store (with an embedding).

Why this order? Create the durable records (Jira/Notion) *before* notifying humans, so the Slack message can include real links and nobody is paged about a ticket that doesn't exist yet.

5.3 Review Manager (Agent 2) — `agents/review_manager/consumer.py`

Subscribes to **both** `dev.pr.updates` and `dev.pr.decisions` . Branches on `event` :

- `PR_OPENED / SYNCHRONIZE / REOPENED` → `find_reviewer` (Senior, Active, not the author) → Slack the reviewer + announce in `#all-kaos` → move Notion to "To be reviewed", Jira to "In Progress".
- `REVIEW_SUBMITTED + CHANGES_REQUESTED` → DM the **author** with the reviewer's comments → public nudge → Notion "Needs attention". (Handling the "Loop of Despair" — rejections.)
- `REVIEW_SUBMITTED + APPROVED` → congratulate the author → "ready to merge". *Crucially it does NOT mark Jira Done here.*
- `PR_MERGED` → Notion "Deploying", Jira stays "In Progress".

Key business rule (encoded in the comments): *Jira only moves to Done when production succeeds, not when the PR merges.* This reflects a real principle — merged ≠ deployed ≠ working. The "Resolved" state is owned exclusively by the Ops Manager after a green prod deploy. This separation is *why* the agents are split by phase.

5.4 Ops Manager (Agent 3) — `agents/ops_manager/main.py` (LLM-driven)

This one **is** an LLM agent (`BaseAgent`) with a system prompt describing two branches:

On `SUCCEEDED` : Jira → comment + transition to "Done"; Notion → "Resolved"; broadcast to `#all-kaos` ; log a `DEPLOYMENT_REPORT` . *This is the only place the loop is allowed to close successfully.*

On `FAILED` :

1. `analyze_deployment_failure` — pulls logs and asks **AWS Bedrock (Titan)** for a friendly root-cause + fix.
2. Jira → "investigating"; Notion → "Needs attention".
3. Personalized Slack DMs to **both** the original author *and* the reviewer (accountability for both sides of the review).
4. `emit_quality_report` — publishes a new `QualityReport` back to `system.quality.reports` with `suggested_assignee = author` . **This is the closed loop.** The failure re-enters triage as a fresh, pre-assigned bug, and the whole cycle restarts automatically.
5. Log a `DEPLOYMENT_REPORT` .

Why use an LLM here specifically? Log diagnosis is genuinely open-ended — you can't pre-program every failure mode. The system prompt constrains *what* to do (the steps), and the model handles the *judgement* (reading logs, phrasing the fix). The deterministic agents handle the mechanical parts; the LLM handles the analytical part.

5.5 Chatbot (Agent 4) — `agents/chatbot/main.py` (LLM-driven, FastAPI :8001)

A FastAPI `/chat` endpoint backed by a `BaseAgent` with five retrieval tools:

- `get_bug_timeline` — reconstructs a human-readable **narrative** from the event store ("🐛 bug reported → 🎫 ticket created → 👤 reviewer assigned → ✅ approved → 🚀 deployed").
- `search_events` — keyword search over event details.
- `find_team_info` — owner/contributors from Neo4j.
- `get_jira_status` — live Jira status for a service.
- `get_bug_solution_details` — pulls the actual PR diff/files from GitHub + Jira resolution comments to explain *how* a bug was fixed.

This is effectively **RAG over your ops history** — the "documents" are events and graph nodes, retrieved by tools and synthesized by the model. It turns the audit log into something a human can interrogate in natural language.

5.6 Control Plane (`agents/control_plane/`) — Mission Control

FastAPI (:8080) + vanilla JS dashboard. Three jobs:

1. **Process orchestration** (`process_manager.py`) — start/stop each agent as a child `subprocess`, tracked by PID, killed cleanly via `psutil` (including child processes). It injects `PYTHONPATH` and `PYTHONUNBUFFERED=1` so logs stream live.
2. **Simulation** — `/api/simulate/*` endpoints forge realistic Sentry/GitHub/deploy payloads and POST them to the Ingestion agent, so you can drive the entire pipeline from buttons (essential for demos without real integrations firing).
3. **UI** — serves the static frontend, polls `/api/status` every 2s, and embeds the chatbot. The frontend even monkey-patches `console.log` to render logs in-page.

Why build this? A distributed, multi-process event system is hard to demo and debug from a terminal. The Control Plane makes the invisible visible and gives a single pane of glass — which for a portfolio/interview project is half the value.

6. The `BaseAgent` — a Hand-Rolled ReAct Loop (`agents/base.py`)

Rather than use LangChain's `AgentExecutor`, KAOS implements its **own ReAct loop**:


```

SystemMessage(instructions) + HumanMessage(task)
for up to 5 iterations:
    response = llm_with_tools.invoke(messages)
    if response has tool_calls:
        for each call: execute tool, append ToolMessage(result)
    else:
        return response.content # done
return "stopped after max iterations"

```

- **LLM:** AWS Bedrock via `langchain_aws.ChatBedrock`, model `amazon.nova-lite-v1:0`, `temperature=0.0` (determinism — you want the same routing decision every time).
- **Tool binding:** `llm.bind_tools(tools)` exposes the `@tool`-decorated functions' schemas to the model; a `tool_map` dispatches calls by name.
- **Hard iteration cap (5):** prevents runaway loops/infinite tool-calling and bounds cost/latency.

Why hand-roll it? Three reasons you can defend:

1. **Transparency/control** — every step is visible and debuggable; no framework magic hiding why a tool fired.
2. **Minimal dependency surface** — you're not pinned to a fast-moving `AgentExecutor` API.
3. **Determinism + guardrails** — explicit max-iterations and `temperature=0` give predictable, bounded behavior, which matters when each tool call has real side effects (creating tickets, sending DMs).

7. State & Memory: the SQLite Event Store (`shared/logger.py`)

A SQLAlchemy layer (defaults to `sqlite:///kaos_events.db`, swappable to Postgres via `DATABASE_URL`) with three tables:

- **pr_events** — the append-only **event log**: `event_type`, `actor`, `repo`, `pr_id`, `details` (JSON), `embedding` (JSON). This is the source of truth for the chatbot's history.
- **notion_tickets** / **jira_tickets** — **linkage tables** mapping a service → its live Notion page-id / Jira issue-key + last-known status.

Why a second store when we already have Kafka + Jira + Notion?

- **Idempotency / dedup:** Before creating a ticket, agents ask "is there already an active Jira ticket for this service?" (`get_active_jira_ticket`). Without this local index you'd spam duplicate tickets on every recurring error. This is the mechanism that makes the closed loop safe.
- **Fast cross-system lookup:** "Find the Notion page for PaymentService" is one indexed query here, versus an expensive search against Notion's API every time.
- **Queryable history for the chatbot:** Kafka is a stream, not a query engine. SQLite gives `get_bug_timeline` and `search_events` cheap reads.

Embeddings: `log_event` lazily loads `sentence-transformers` (`all-MiniLM-L6-v2`) and stores a vector for each event's text. The *intent* is semantic search over incidents ("have we seen anything like this NPE before?"). **Honest current state:** the vector is persisted but `search_events` still uses SQL `LIKE`; true vector similarity search isn't wired up yet (a real production version would move to Postgres

+ `pgvector`). Good to acknowledge proactively — it shows you know the difference between "stored embeddings" and "a working semantic index."

8. Cross-Cutting Concerns

8.1 Configuration (`config/settings.py`)

Pydantic-Settings `BaseSettings` loads everything from `.env` with typed fields and sane defaults. Secrets (Kafka SASL, Neo4j, AWS, Slack, Jira, Notion, GitHub) are never hard-coded. Optional integrations are typed `str | None` so the app boots even if, say, GitHub isn't configured (the chatbot degrades to "simulation mode").

8.2 Resilience (`shared/utils/retries.py`)

A `retry_with_backoff` decorator (exponential backoff + jitter) wraps the network-bound Triager tools (Neo4j, Notion, Jira, Slack). External APIs fail transiently; blind retries cause thundering herds, so the jitter (`random.uniform(0,1)`) spreads them out. After N attempts it re-raises so failures surface rather than hang silently.

8.3 Structured logging (`shared/logger.py :: JsonFormatter`)

All logs are emitted as JSON (timestamp, level, logger, module, funcName, lineNo). **Why JSON?** In a multi-process distributed system, you ship logs to an aggregator (CloudWatch/ELK) and need to filter/query them — structured logs are machine-parseable; pretty-printed strings aren't.

8.4 Severity / status mapping

A recurring pattern: external vocabularies are normalized to internal enums at the boundary (Sentry `fatal` → `CRITICAL` ; severity→Jira priority; `success/failure` → `SUCCEEDED/FAILED`). Centralizing these maps keeps downstream logic clean.

9. The End-to-End Flow (the "money" diagram)



The defining feature is that **last arrow**: a production failure doesn't just alert someone — it becomes a new, pre-assigned bug report, and the entire automated lifecycle runs again. The system is *self-healing-ish*: it keeps cycling work back to the right person until prod is green.

10. Honest Limitations (say these before the interviewer finds them)

These are demo/portfolio simplifications, not oversights — knowing them is a strength:

1. **Notion dedup is intentionally disabled** ("DEMO HACK" comment) so each run creates a fresh row.

2. `check_pr_status` always returns **CLEAN** — merge-conflict detection is stubbed; the architecture describes polling the GitHub API but the tool is simulated.
3. **Bedrock log analysis uses hard-coded sample logs**, not the real `logs_url`.
4. **Embeddings are stored but not searched** — semantic search is SQL `LIKE` today.
5. **Auto-commit offsets** = at-least-once, so a crash mid-processing could re-run side effects (mitigated by the SQLite idempotency checks, but not bulletproof).
6. **Duplicate agent entry points** (`main.py` vs `consumer.py`) — the deterministic `consumer.py` paths are the live ones.
7. `temperature=0` + **5-iteration cap** bound the LLM, but there's no structured validation of tool *arguments* beyond Pydantic tool schemas.

11. One-Line Summary of Every Component

Component	File(s)	One-liner
Ingestion	<code>agents/ingestion/</code>	Webhook→Kafka anti-corruption layer (FastAPI :8000)
Triager	<code>agents/triager/consumer.py</code>	Routes bugs to active owners via graph + files Jira/Notion/Slack
Review Manager	<code>agents/review_manager/consumer.py</code>	Assigns reviewers, handles approve/reject, tracks PR→merge
Ops Manager	<code>agents/ops_manager/main.py</code>	LLM agent: diagnoses deploy failures, closes the loop
Chatbot	<code>agents/chatbot/</code>	RAG-style Q&A over event store + graph (FastAPI :8001)
Control Plane	<code>agents/control_plane/</code>	Subprocess orchestration + simulation + UI (FastAPI :8080)
BaseAgent	<code>agents/base.py</code>	Hand-rolled ReAct loop over Bedrock
Kafka client/consumer	<code>shared/kafka/</code>	Confluent factory + Template-Method consumer base
Neo4j client	<code>shared/neo4j/client.py</code>	Thin Cypher executor
Event store	<code>shared/logger.py</code>	SQLAlchemy event log + ticket linkage + embeddings
Settings	<code>config/settings.py</code>	Typed env config via Pydantic-Settings
Retries	<code>shared/utils/retries.py</code>	Exponential backoff + jitter decorator